ЛР 2. HTTP API и обработка данных на Flask

Дисциплина:«Протоколы и интерфейсы информационных систем».

Продолжение проекта it-service-desk. В первой лабораторной был подготовлен интерфейс Angular. Теперь необходимо реализовать серверный HTTP API на Flask и обработку заявок в памяти процесса.

> 💡 Что вы сделаете в лабораторной работе
>
> продолжите работу с тем же репозиторием it-service-desk;
>
> • реализуете GET-запросы и фильтрацию заявок;
>
> • добавите проверку query-параметров и JSON-тела;
>
> • реализуете POST, PATCH и DELETE;
>
> • проверите HTTP-коды 200, 201, 204, 400 и 404;
>
> • проследите обработку запроса с помощью debugger;
>
> • подготовите backend к переходу на MySQL в следующей лабораторной.

1. Перед практикой минимум теории

1.1. HTTP, ресурс и операция

URL обозначает ресурс, а HTTP-метод — действие над ним. GET читает данные, POST создаёт ресурс, PATCH изменяет часть полей, DELETE удаляет. GET не должен изменять состояние.

Ситуация	Код	Смысл
Успешное чтение/изменение	200	Запрос выполнен
Создание ресурса	201 + Location	Ресурс создан
Удаление	204	Успех без тела ответа
Ошибка запроса	400	Некорректные данные
Ресурс отсутствует	404	Объект не найден
Неверный Content-Type	415	Сервер не принимает тип тела
Сбой сервера	500	Непредвиденная ошибка
Заглушка шаблона	501	Шаг ещё не реализован

JSON — формат тела, а не протокол. В запросе укажите *Content-Type: application/json*. Сервер не должен доверять интерфейсу: запрос можно отправить вообще без Angular.

```
# пример HTTP-запроса
POST /api/tickets
Content-Type: application/json

{"title":"Не печатает принтер","equipmentId":104}
```

1.2. Flask и слои

Factory create_app создаёт приложение; *Blueprint* группирует маршруты. Маршрут читает HTTP-запрос, валидатор проверяет значения, репозиторий работает с данными. В этой ЛР *STORAGE=memory* выбирает *MemoryRepository*; режим *mysql* будет использован в следующей работе.

> 💡 Цепочка обработки
>
> HTTP-запрос → Flask route → validation.py → MemoryRepository → HTTP-ответ.
>
> Сервер проверяет данные независимо от Angular: запрос можно отправить напрямую.

Типы и копии

Список Python существует только пока работает сервер. После перезапуска начальные данные создаются заново. ID назначается серверным счётчиком и после удаления не переиспользуется. Для *equipmentId* используется *type(value) is int*, поскольку bool в Python является подклассом int.

> ! Контрольный вопрос
>
> Какой ответ нужен при POST с пустым title, GET отсутствующей заявки и DELETE существующей заявки? Почему оборудование нельзя проверять только в выпадающем списке браузера?

2. Подготовка проекта продолжение работы с репозиторием

> ! ПРАКТИЧЕСКОЕ ЗАДАНИЕ - Продолжить работу с существующим репозиторием
>
> 1. Откройте тот же каталог it-service-desk, который использовался в ЛР №1. Не клонируйте шаблон заново.
> 2. Активируйте backend/.venv.
> 3. Проверьте backend/.env: должно быть STORAGE=memory.
> 4. Запустите starter-тест.
> 5. Запустите Flask на порту 5000 и оставьте сервер работать.
>
> **Результат:** GET /api/health возвращает 200 и storage=memory; /api/tickets до реализации возвращает 501.

Откройте **тот же** it-service-desk. Не клонируйте шаблон заново. Из корня активируйте окружение:

```
# it-service-desk
Windows cmd — backend\.venv\Scripts\activate.bat;
PowerShell — .\backend\.venv\Scripts\Activate.ps1;
macOS/Linux — source backend/.venv/bin/activate.
```

Проверьте backend/.env: STORAGE=memory. Затем:

```
# it-service-desk/backend
cd backend
python -m pytest tests/test_starter.py -q
python -m flask --app app:create_app run --debug --port 5000
```

В терминале активируйте окружение и оставайтесь в корне:

```
#  it-service-desk
python tools/request.py GET /api/health
python tools/request.py GET /api/tickets
```

> 💡 Важно
>
> Утилита tools/request.py печатает HTTP-код, Location и тело ответа.
>
> JSON-файлы передаются через --data, поэтому не требуется разбираться с различиями кавычек curl в PowerShell и bash.

3. Практика разработка HTTP API

3.1. Прочитайте контракт

> ! ПРАКТИЧЕСКОЕ ЗАДАНИЕ - Сопоставить клиентскую модель и серверный контракт
>
> 1. Откройте docs/api-contract.md.
> 2. Сравните контракт с Angular-моделью в frontend/src/app/models.
> 3. Откройте backend/app/memory.py и найдите начальные данные.
> 4. Обратите внимание: equipmentId приходит от клиента; equipmentName, inventoryNumber, createdAt и updatedAt формирует сервер.
> 5. До программирования запишите ожидаемые результаты запросов из таблицы ниже.
>
> **Результат:** Вы понимаете, какие поля принимает сервер, какие формирует сам и какие ответы должны получиться.

Запишите ожидаемые результаты:

Запрос	Ожидается после выполнения ЛР
GET /api/tickets	Массив из 3 начальных заявок
GET /api/tickets?scope=open	2 незакрытые
GET /api/tickets?status=new	1 новая
GET /api/tickets?q=КОМПЬЮТЕР	Поиск без регистра
GET /api/tickets?status=open	400: open — режим, не статус
GET /api/tickets/999	404

3.2. Реализуйте чтение из памяти

> **!** ПРАКТИЧЕСКОЕ ЗАДАНИЕ — Реализовать list() и get()
>
> 1. Откройте backend/app/memory.py.
> 2. Замените только методы list и get, сохранив остальной класс.
> 3. Сначала применяйте точный status, затем scope, затем текстовый поиск q.
> 4. Для q используйте casefold(), для scope=open исключите closed.
> 5. Верните новые ID первыми с помощью reversed().
> 6. Методы находятся внутри класса: сохраните корректный отступ.
>
> **Результат:** GET /api/tickets и варианты фильтрации возвращают ожидаемые массивы; отсутствующий ID приводит к 404.

```
# backend/app/memory.py
def list(self, status="", scope="all", q=""):
    result = self.items
    if status:
        result = [item for item in result if item["status"] == status]
    if scope == "open":
        result = [item for item in result if item["status"] != "closed"]
    if q:
        result = [item for item in result if q.casefold() in item["title"].casefold()]
    return [self._decorate(item) for item in reversed(result)]
def get(self, ticket_id):
    return self._decorate(self._item(ticket_id))
```

> **!** Контрольный вопрос

> Почему фильтры status, scope и q в этой реализации соединяются как AND? Что должна вернуть функция, если ни одна заявка не подходит?

3.3. Проверьте параметры запроса

> **!** ПРАКТИЧЕСКОЕ ЗАДАНИЕ — Реализовать ticket_query()
>
> 1. Откройте backend/app/validation.py.
> 2. Замените функцию ticket_query целиком.
> 3. Разрешите только status, scope и q.
> 4. Запретите повторяющиеся query-параметры.
> 5. Проверьте status по STATUSES.
> 6. Для scope разрешите только all и open; q после strip должен быть не длиннее 120 символов.
>
> **Результат:** Корректные параметры передаются в репозиторий, неизвестные/повторяющиеся значения дают 400.

```
# backend/app/validation.py
def ticket_query(args):
    if set(args) - {"status", "scope", "q"} or any(len(args.getlist(k)) != 1 for k in args):
        raise ApiError("VALIDATION_ERROR", "Неизвестные или повторяющиеся параметры
запроса.")
    status = args.get("status", "")
    scope = args.get("scope", "all")
    query = args.get("q", "").strip()
    if status and status not in STATUSES:
        raise ApiError("VALIDATION_ERROR", "Недопустимый статус фильтра.")
    if scope not in ("all", "open") or len(query) > 120:
        raise ApiError("VALIDATION_ERROR", "Некорректные параметры поиска.")
    return {"status": status, "scope": scope, "q": query}
```

request.args — *MultiDict*: параметр может встретиться несколько раз. Здесь повторы запрещены, чтобы сервер не выбирал одно из противоречивых значений молча. Q ограничен по длине.

В *backend/app/routes.py* замените тела маршрутов *tickets* и *ticket*. Декораторы уже есть; если вставляете фрагмент с декоратором, заменяйте старый блок целиком, не дублируйте его.

> **!** ПРАКТИЧЕСКОЕ ЗАДАНИЕ — Подключить GET-маршруты
>
> 1. Откройте backend/app/routes.py.
> 2. Замените тела существующих маршрутов tickets и ticket.
> 3. Не дублируйте декораторы: если копируете блок целиком, удалите старый.

4. Повторите запросы из таблицы раздела 3.1.

Результат: 501 исчезает для GET-маршрутов; фильтры и 404 работают согласно контракту.

```
# backend/app/routes.py
@api.get("/tickets")
def tickets():
    return repo().list(**ticket_query(request.args))
@api.get("/tickets/<int:ticket_id>")
def ticket(ticket_id):
    return repo().get(ticket_id)
```

Теперь повторите запросы из шага 2. Если ещё получаете 501, прочитайте поле message и проверьте,
что сохранён нужный файл. При режиме debug сервер перезагружает код автоматически.

3.4. Реализуйте валидацию тела

В *backend/app/validation.py* замените ticket_payload. Сначала прочитайте требования.

💡 Требования к телу запроса

- JSON должен быть непустым объектом; массив, null и неизвестные поля запрещены.
- POST: обязательны title и equipmentId; description по умолчанию пустая строка.
- title после trim: 3–120 символов; description: до 2000.
- PATCH принимает только переданную часть полей и может изменять status.
- При POST status всегда назначает сервер как new.

! ПРАКТИЧЕСКОЕ ЗАДАНИЕ — Реализовать ticket_payload()

1. Откройте backend/app/validation.py.
2. Замените заглушку ticket_payload целиком.
3. Сначала проверьте тип объекта и неизвестные поля.
4. Затем проверьте обязательные поля POST.
5. Для строк выполните strip до проверки длины.
6. Проверьте equipmentId через type(value) is int и value >= 1.
7. При partial=False установите description по умолчанию и status='new'.

Результат: tests/test_validation.py проходит; некорректное тело даёт 400, а не 500.

Функция проходит эти условия сверху вниз. Вставьте её целиком вместо заглушки:

```python
# backend/app/validation.py
def ticket_payload(data, partial=False):
    if not isinstance(data, dict):
        raise ApiError("VALIDATION_ERROR", "Ожидается JSON-объект.")
    allowed = {"title", "description", "equipmentId"}
    if partial:
        allowed.add("status")
    if set(data) - allowed or not data:
        raise ApiError("VALIDATION_ERROR", "Переданы неизвестные поля или пустой объект.")
    if not partial and not {"title", "equipmentId"} <= set(data):
        raise ApiError("VALIDATION_ERROR", "Укажите название и оборудование.")
    result = {}
    for key in ("title", "description"):
        if key not in data:
            continue
        value = data[key]
        if not isinstance(value, str):
            raise ApiError("VALIDATION_ERROR", "Текстовые поля должны быть строками.")
        value = value.strip()
        minimum, maximum = (3, 120) if key == "title" else (0, 2000)
        if not minimum <= len(value) <= maximum:
            raise ApiError("VALIDATION_ERROR", f"Длина {key}: от {minimum} до {maximum}
символов.")
        result[key] = value
    if "equipmentId" in data:
        value = data["equipmentId"]
        if type(value) is not int or value < 1:
            raise ApiError("VALIDATION_ERROR", "Выберите оборудование из списка.")
        result["equipmentId"] = value
    if "status" in data:
        if data["status"] not in STATUSES:
            raise ApiError("VALIDATION_ERROR", "Недопустимый статус заявки.")
        result["status"] = data["status"]
    if not partial:
        result.setdefault("description", "")
        result["status"] = "new"
    return result
```

> **!** Контрольный вопрос
>
> Почему неизвестные поля проверяются до обработки значений? Почему strip выполняется до проверки длины? Зачем PATCH вызывает валидатор с partial=True?

Запустите:

```
# it-service-desk/backend
backend python -m pytest tests/test_validation.py -q
```

Ожидается успешный запуск всех параметризованных случаев. До замены тела валидатора эти тесты не были проверкой готовности среды: их падение означало невыполненное задание.

3.5. Создайте заявку

В MemoryRepository замените create. Сначала проверяется оборудование, затем назначается ID, после записи создаётся событие. В этой ЛР всё ещё память процесса, не транзакция БД.

> **!** ПРАКТИЧЕСКОЕ ЗАДАНИЕ — Реализовать создание заявки
>
> 1. В backend/app/memory.py замените create.
> 2. Сначала проверьте существование оборудования через _device.
> 3. Назначьте timestamp, ID и status='new'.
> 4. Добавьте запись в items и событие created.
> 5. В backend/app/routes.py замените create_ticket.
> 6. Верните 201 и заголовок Location.
>
> **Результат:** POST создаёт заявку, возвращает status=new и Location вида /api/tickets/<id>.

```
# backend/app/memory.p
def create(self, values):
    self._device(values["equipmentId"])
    timestamp = now()
    item = {**values,"id":self.next_id,"status":"new",
            "createdAt":timestamp,"updatedAt":timestamp}
    self.next_id += 1
    self.items.append(item)
    self._record(item["id"],"created","Заявка зарегистрирована")
    return self._decorate(item)
```

В *routes.py* замените *create_ticket:*

```
# backend/app/routes.py
@api.post("/tickets")
def create_ticket():
    result = repo().create(ticket_payload(request.get_json()))
    return result, 201, {"Location": url_for("api.ticket", ticket_id=result["id"])}
```

В *requests/create-ticket.json* уже есть корректное тело. Во втором терминале из корня:

```
# it-service-desk
python tools/request.py POST /api/tickets --data requests/create-ticket.json
```

> 💡 Проверка
>
> Ожидаются HTTP 201, Location и status=new.
>
> Запишите фактический ID. При повторных запусках он может отличаться.
>
> Во всех следующих командах используйте ID из собственного ответа.

3.6. Измените и удалите запись

В *memory.py* замените *update и delete*. Перед изменением проверьте новую связь equipmentId, чтобы ошибочный запрос не успел изменить запись частично.

> ! ПРАКТИЧЕСКОЕ ЗАДАНИЕ — Реализовать update() и delete()
>
> 1. В backend/app/memory.py замените update и delete.
> 2. Перед изменением equipmentId сначала проверьте новую связь через _device.
> 3. После update обновите updatedAt и создайте событие updated.
> 4. При delete удалите заявку и связанные события.
> 5. В backend/app/routes.py подключите PATCH и DELETE.
>
> **Результат:** PATCH возвращает изменённую заявку; DELETE возвращает 204 без тела; последующий GET возвращает 404.

```
# backend/app/memory.py
def update(self, ticket_id, values):
    item = self._item(ticket_id)
    if "equipmentId" in values:
        self._device(values["equipmentId"])
    item.update(values)
    item["updatedAt"] = now()
    self._record(ticket_id,"updated","Обновлена заявка")
    return self._decorate(item)
def delete(self, ticket_id):
    item = self._item(ticket_id)
    self.items.remove(item)
    self.history = [event for event in self.history if event["ticketId"] != ticket_id]
```

В *routes.py* замените соответствующие блоки:

```
# backend/app/routes.py
@api.patch("/tickets/<int:ticket_id>")
def update_ticket(ticket_id):
    return repo().update(ticket_id, ticket_payload(request.get_json(), partial=True))
@api.delete("/tickets/<int:ticket_id>")
def delete_ticket(ticket_id):
    repo().delete(ticket_id)
    return "", 204
```

Пример для ID 4; замените его своим:

```
it-service-desk
python tools/request.py PATCH /api/tickets/4 --data requests/update-ticket.json
python tools/request.py GET /api/tickets/4/events
python tools/request.py DELETE /api/tickets/4
python tools/request.py GET /api/tickets/4
```

> 💡 Ожидаемая последовательность
>
> PATCH → 200 и status=in_progress.
>
> GET events → события создания и изменения.
>
> DELETE → 204 без тела.
>
> GET после удаления → 404. Повторный DELETE → 404.

4. Надёжность и отладка

> ! ПРАКТИЧЕСКОЕ ЗАДАНИЕ — Проверить некорректные данные
>
> 1. Отправьте requests/invalid-ticket.json.
> 2. Создайте рядом тестовый JSON с equipmentId=999 и отправьте POST.
> 3. Убедитесь, что оба запроса дают 400, а не 500.
> 4. После ошибок проверьте, что количество заявок не изменилось.
>
> **Результат:** Некорректные запросы не повреждают состояние хранилища.

> ! ПРАКТИЧЕСКОЕ ЗАДАНИЕ — Проследить POST в debugger
>
> 1. Остановите обычный Flask-сервер, чтобы освободить порт 5000.
> 2. В VS Code выберите интерпретатор backend/.venv и конфигурацию «ЛР2: Flask».

3. Поставьте breakpoint в create_ticket перед вызовом репозитория.
4. Отправьте POST и изучите словарь после валидации.
5. Выполните Step into и проследите назначение ID.

Результат: Вы можете объяснить путь данных от HTTP-запроса через валидацию до MemoryRepository.

❗ ПРАКТИЧЕСКОЕ ЗАДАНИЕ — Проверить ограничение хранения в памяти

1. Создайте новую заявку.
2. Остановите Flask.
3. Запустите Flask снова.
4. Выполните GET /api/tickets.

Результат: Созданная заявка исчезает, а начальных записей снова три. Это ожидаемо и будет устранено в ЛР №3 с БД.

5. Итоговая самопроверка

💡 Чек-лист

/api/health возвращает 200 и storage=memory.

GET списка, фильтры, поиск и GET по ID работают.

Некорректные query-параметры дают 400.

Валидация JSON-тела проходит tests/test_validation.py.

POST возвращает 201 + Location.

PATCH изменяет только переданные поля.

DELETE возвращает 204 без тела, а повторный запрос — 404.

Некорректный equipmentId не изменяет данные.

Вы можете объяснить, почему данные исчезают после перезапуска.

6. Что сдавать

💡 Перечень

1. Проект it-service-desk с выполненной ЛР №2.
2. Реализованные memory.py, validation.py и routes.py.
3. Успешные тесты валидации и демонстрация основных HTTP-запросов.

4. Ответы на контрольные вопросы.

7. Материалы для подготовки

1. Flask Quickstart: Routing, HTTP Methods, JSON.
2. Flask Testing: test client и JSON.
3. MDN HTTP: запрос, ответ, методы и коды состояния.
4. Python venv: создание и активация виртуального окружения.